UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

TFG del Grado en Ingeniería Informática

título del TFG

Presentado por Ángel Ortiz de Lejarazu Sánchez
en Universidad de Burgos — 9 de mayo de 2026
Tutor: nombre tutor

UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Ángel Ortiz de Lejarazu Sánchez, con DNI 32702325M, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 9 de mayo de 2026

Vº. Bº. del Tutor:	Vº. Bº. del co-tutor:
D. nombre tutor	D. nombre co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general iii

Índice de figuras v

Índice de tablas vi

1. Introducción 1
- 1.1. La crisis de accesibilidad y la necesidad de un triaje frugal . 1
- 1.2. Estado del arte: De los enfoques centralizados a la IA sostenible 2
- 1.3. Justificación tecnológica y enfoque del proyecto 3

2. Objetivos del proyecto 5
- 2.1. Objetivos Generales 5
- 2.2. Objetivos funcionales 6
- 2.3. Objetivos no funcionales 8
- 2.4. Objetivos personales 9

3. Conceptos teóricos 11
- 3.1. Clasificación supervisada en texto clínico 12
- 3.2. Preprocesamiento lingüístico y normalización 12
- 3.3. Representación vectorial mediante TF-IDF 13
- 3.4. Reducción de dimensionalidad con TruncatedSVD 14
- 3.5. Clasificación mediante Máquinas de Vector de Soporte . . . 15
- 3.6. Clasificación mediante Perceptrón Multicapa 16
- 3.7. Optimización de hiperparámetros y evaluación 17
- 3.8. Decisión con umbral de confianza y derivación a atención general 18

3.9. Captura de síntomas por voz 19
3.10. IA Frugal y Edge AI 19
3.11. Conclusiones del capítulo 20

4. Técnicas y herramientas **21**

5. Aspectos relevantes del desarrollo del proyecto **23**

6. Trabajos relacionados **25**

7. Conclusiones y Líneas de trabajo futuras **27**

Bibliografía **33**

Índice de figuras

Índice de tablas

1. Introducción

1.1. La crisis de accesibilidad y la necesidad de un triaje frugal

La medicina actual vive una paradoja cada vez más evidente: mientras los avances terapéuticos y diagnósticos han alcanzado un grado de complejidad y desarrollo sin precedentes, el acceso básico al sistema sanitario se ha deteriorado hasta convertirse en un problema operativo de primer orden. En este contexto, la denominada "puerta de entrada" al sistema —los procesos de triaje y asignación de citas— se ha transformado en un cuello de botella que afecta tanto a la seguridad y la privacidad del paciente como a la eficiencia de las instituciones sanitarias.

Los datos correspondientes al periodo 2024–2025 describen un escenario de saturación estructural. En el contexto español, el Barómetro Sanitario indica que aproximadamente el 70 % de los pacientes debe esperar más de un día para ser atendido en Atención Primaria, con demoras medias que oscilan entre los 8,7 y los 9,78 días. Esta latencia desvirtúa el carácter preventivo del primer nivel asistencial y deriva en un desplazamiento de la demanda hacia los servicios de urgencias, que terminan absorbiendo patologías de baja complejidad [22, 26, 33]. De forma paralela, en Estados Unidos el tiempo medio de espera para una primera cita médica se ha incrementado de manera sostenida, alcanzando un máximo histórico de 31 días en 2025 [32].

Este colapso asistencial se ve agravado por la falibilidad inherente al triaje manual. La literatura científica reciente señala que la clasificación inicial de la demanda clínica presenta tasas de error relevantes, oscilando entre el 10 % y el 19 % tanto para el sobre-triaje —que implica un uso ineficiente de recursos— como para el infra-triaje, que conlleva riesgos

clínicos por retrasos en la atención [23]. En este escenario, la automatización del procesamiento de síntomas expresados en lenguaje natural deja de ser una mera mejora tecnológica para convertirse en una necesidad operativa crítica para la sostenibilidad del sistema [8].

1.2. Estado del arte: De los enfoques centralizados a la IA sostenible

La respuesta tecnológica dominante ante esta crisis ha estado marcada por la adopción de modelos cada vez más complejos y de gran escala, junto con una fuerte dependencia del *cloud computing*. Sin embargo, un análisis crítico de las soluciones actuales revela limitaciones estructurales que justifican la exploración de enfoques alternativos.

El caso de Babylon Health, empresa referente en salud digital que se declaró en bancarrota en 2023, ilustra los riesgos de modelos opacos ("cajas negras") que carecen de una validación clínica robusta frente a la variabilidad del mundo real [10, 12]. Por otro lado, plataformas consolidadas como Ada Health, aunque ofrecen una elevada precisión diagnóstica, presentan una arquitectura centralizada dependiente de la conectividad a internet [25, 7]. Esta dependencia no solo limita su uso en zonas rurales o con infraestructura deficiente, sino que introduce vectores de riesgo en la privacidad al requerir la transmisión de datos sensibles a servidores externos [4, 17].

Asimismo, la irrupción de Modelos de Lenguaje Grande (LLMs) como GPT-4 plantea desafíos de sostenibilidad. Estudios recientes advierten que el coste computacional y energético de estos modelos dificulta su despliegue masivo en la sanidad pública, configurando lo que se ha denominado "IA Roja". A esto se suma el riesgo de "alucinaciones" y la exposición de datos privados a terceros [19, 24, 16].

Frente a este panorama, este Trabajo de Fin de Grado se alinea con el paradigma de la IA Frugal (*Frugal AI*) y la computación en el borde (*Edge AI*). La IA frugal busca maximizar el valor funcional minimizando el consumo de recursos computacionales y energéticos [1, 29]. Por su parte, el *Edge AI* permite ejecutar la inferencia directamente en el dispositivo local, reduciendo la latencia y garantizando que los datos médicos nunca salgan del entorno del paciente, devolviendo así la soberanía del dato a la institución y al usuario [4, 17].

1.3. Justificación tecnológica y enfoque del proyecto

Con el objetivo de dar respuesta a los problemas identificados, este trabajo propone el diseño, desarrollo y validación de una aplicación de triaje clínico *stand alone*, basada en la combinación sinérgica de tres técnicas consolidadas: TF-IDF, SVD y SVM. Esta arquitectura responde a una estrategia de ingeniería orientada a la eficiencia y la explicabilidad:

- **TF-IDF (*Term Frequency – Inverse Document Frequency*):** se emplea como mecanismo de filtrado semántico para identificar términos clínicamente relevantes (ej. "disnea", "dolor torácico") dentro del ruido del lenguaje natural, ponderando la especificidad sobre la frecuencia [3].
- **SVD (*Singular Value Decomposition*):** se incorpora para abordar la sinonimia y polisemia médica. Mediante la reducción de dimensionalidad y la extracción de "conceptos latentes", permite al sistema reconocer patrones patológicos incluso cuando el vocabulario del paciente no coincide exactamente con el del entrenamiento, optimizando el rendimiento computacional [20].
- **SVM (*Support Vector Machines*):** se selecciona como motor de decisión por su robustez matemática en espacios de alta dimensionalidad y la claridad de sus fronteras de decisión (hiperplanos). A diferencia de las redes neuronales profundas, las SVM ofrecen un equilibrio superior entre precisión y transparencia, facilitando la auditoría clínica de las decisiones [9].

El alcance de este Trabajo de Fin de Grado trasciende el análisis teórico para materializarse en un prototipo funcional. El objetivo principal es demostrar que un sistema ligero, privado y operativo sin conexión a internet puede clasificar eficazmente la demanda asistencial. Este enfoque aspira a contribuir a la reducción de la carga administrativa, la homogeneización del triaje y la asignación eficiente de citas, todo ello bajo un marco de privacidad por diseño y sostenibilidad tecnológica.

Pruebas : Glosario USo : de otros CoT debido TAmbién *Interfaz Gráfica de Usuario* (GUI) y nos aprece todo

2. Objetivos del proyecto

2.1. Objetivos Generales

Desarrollar y validar un **prototipo stand alone** en Python capaz de **clasificar síntomas descritos en lenguaje natural** y **asignar una categoría/especialidad médica orientada a la gestión de citas**, manteniendo la ejecución **local** (sin dependencia de servicios externos) y priorizando criterios de **eficiencia, trazabilidad y privacidad por diseño**.

El objetivo se concreta en:

- Construir un sistema ejecutable que incorpore un flujo completo de NLP (preprocesado → representación → entrenamiento → predicción).
- Implementar y comparar, bajo un protocolo homogéneo, tres enfoques: *Term Frequency–Inverse Document Frequency* (TF-IDF) (base), TF-IDF + *Singular Value Decomposition* (SVD) (reducción de dimensionalidad para capturar estructura latente) y *Support Vector Machine* (SVM) como clasificador supervisado.
- Evaluar el rendimiento predictivo y el rendimiento operativo (tiempos, memoria) para determinar el **mejor equilibrio práctico** entre precisión y coste computacional.
- Demostrar su **aplicabilidad** como componente autónomo de apoyo al triaje/derivación de demanda en escenarios con restricciones de conectividad y con necesidad de minimizar exposición de datos.

2.2. Objetivos funcionales

Los objetivos funcionales describen **qué hace el sistema** y se formulan como metas técnicas **medibles y verificables**.

F1. Ingesta, validación y preparación del conjunto de datos

- Definir el esquema operativo de datos (`texto_síntomas` → `categoría`) y desarrollar carga desde fichero local (p. ej., CSV), con validaciones automáticas: valores nulos, duplicados, etiquetas fuera de catálogo y control de distribución de clases.
- Dividir los datos siempre del mismo modo (entrenamiento/prueba y, cuando proceda, validación cruzada estratificada) para que todos los modelos se evalúen con las mismas particiones y la comparación sea justa.

F2. Pipeline completo de procesamiento de lenguaje natural (NLP)

- Implementar un pipeline configurable de preprocesado (normalización, limpieza, tokenización y n-gramas; lematización/stemming solo si aporta mejora cuantificable).
- Garantizar consistencia entre entrenamiento y predicción mediante persistencia de transformaciones (vectorizador, normalización y parámetros).

F3. Modelo base basado en TF-IDF

- Implementar una línea base con TF-IDF como representación y un clasificador supervisado apropiado para texto (p. ej., Regresión logística o Naive Bayes) para establecer una referencia sólida.
- Medir sensibilidad a parámetros clave (rango de n-gramas, `min_df/max_df`, normalización) y documentar su impacto en métricas y coste.

F4. Reducción de dimensionalidad mediante TruncatedSVD

- Integrar TruncatedSVD sobre la matriz TF-IDF y entrenar un clasificador sobre el espacio reducido.

- Definir un criterio de selección del número de componentes (por validación) y cuantificar: (i) variación de rendimiento (F1 macro/micro), (ii) reducción de tiempo/memoria y (iii) robustez ante variaciones lingüísticas.

F5. Clasificador SVM

- Implementar un modelo SVM (preferentemente lineal si el compromiso coste-rendimiento lo justifica) entrenado sobre TF-IDF y, cuando proceda, sobre TF-IDF + SVD.
- Ajustar hiperparámetros (p. ej., `C`, `class_weight`) con validación cruzada y criterios de selección explícitos (p. ej., optimización de macro-F1 en presencia de desbalanceo).

F6. Predicción operativa a partir de texto libre

- Desarrollar una interfaz de predicción (CLI o aplicación ligera) que reciba texto libre y devuelva:
  - Categoría predicha.
  - Indicador de confianza o puntuación (p. ej., margen/score; probabilidad calibrada si es viable).
  - Opcional: top-k categorías con puntuación para facilitar análisis y depuración.

F7. Comparación experimental controlada y análisis por escenarios

- Comparar los enfoques bajo el mismo protocolo, reportando **precisión, Recall (sensibilidad), F1-score (macro y micro)**, matrices de confusión y métricas por clase.
- Evaluar comportamiento ante distintos tipos de entrada: textos muy breves vs. descriptivos, ruido ortográfico, abreviaturas y variabilidad de vocabulario.
- Identificar fortalezas y debilidades prácticas de cada enfoque a partir de errores observados (análisis de fallos y patrones de confusión entre categorías).

2.3. Objetivos no funcionales

Los objetivos no funcionales definen cómo se construye el software para asegurar robustez y aplicabilidad real.

NF1. Arquitectura modular orientada a extensión

- Diseñar una estructura por módulos (datos, preprocesado, características, modelos, evaluación, predicción) con interfaces claras para facilitar mantenimiento y futuras ampliaciones.

NF2. Desarrollo en Python con dependencias acotadas

- Implementar el sistema en Python apoyándose en librerías consolidadas (scikit-learn, numpy, pandas) para maximizar portabilidad y estabilidad del entorno.

NF3. Reproducibilidad y trazabilidad experimental

- Controlar aleatoriedad (semillas), registrar configuración de experimentos y versiones de dependencias, y asegurar que los resultados puedan replicarse en otro equipo.

NF4. Metodología experimental rigurosa

- Aplicar validación cruzada estratificada y un esquema de búsqueda de hiperparámetros (grid/random) con límites de coste realistas.

- Establecer criterios de selección transparentes (métrica objetivo, tratamiento del desbalanceo, calibración si se aplica).

NF5. Stand alone y minimización de exposición de datos

- Garantizar ejecución local sin APIs ni servicios cloud, reduciendo dependencia de internet y favoreciendo privacidad y control del flujo de datos.

NF6. Eficiencia computacional

- Medir y optimizar tiempos de entrenamiento y predicción y consumo de memoria. Justificar decisiones (p. ej., SVM lineal vs. kernel) en función del equilibrio precisión-coste.

NF7. Calidad de código y documentación

- Mantener claridad del código (estructura, nombres, logging), documentación de uso (entrenar, evaluar, predecir) y guía de reproducción de experimentos.
- Incluir pruebas básicas de componentes críticos del pipeline para evitar regresiones.

NF8. Preparación para integración futura con voz

- Desacoplar la capa de entrada para que la fuente de texto sea intercambiable (teclado/fichero/transcripción), dejando definida una interfaz para incorporar reconocimiento de voz en iteraciones futuras sin alterar el núcleo de modelado.

2.4. Objetivos personales

- Consolidar una comprensión aplicada de **clasificación supervisada** en texto clínico, incluyendo errores típicos, desbalanceo y evaluación por clases.
- Comprender de forma operativa la contribución de cada etapa: TF-IDF (representación), SVD (reducción/captura de estructura latente) y SVM (fronteras de decisión), y aprender a decidir cuándo compensan sus costes.
- Aplicar técnicas de NLP a un caso de uso sanitario con restricciones realistas (autonomía, privacidad, eficiencia), diseñando el pipeline de forma reutilizable para otros ámbitos de clasificación de texto y para futuras variantes del canal de entrada (p. ej., transcripción de llamadas telefónicas para la obtención/gestión de citas), evitando soluciones dependientes de servicios externos.

- Culminar el TFG con un **producto funcional demostrable** y una memoria técnica sólida, cumpliendo los requisitos académicos necesarios para **finalizar el Grado en Ingeniería Informática**, facilitando la progresión en la carrera profesional.

3. Conceptos teóricos

Este capítulo recoge los conceptos que sustentan el sistema desarrollado: cómo se modela la asignación automática de especialidades médicas, qué transformaciones se aplican al texto del paciente y por qué se han elegido los algoritmos que componen el pipeline. El objetivo no es enumerar definiciones, sino dejar claro el porqué de cada decisión técnica en un contexto de ejecución local y recursos limitados.

El sistema implementa cuatro modelos que se pueden seleccionar desde la interfaz del paciente. Todos comparten la misma fase de preprocesamiento y representación inicial, pero combinan los componentes posteriores de forma distinta:

- **TF-IDF.** Vectorización TF-IDF seguida de un clasificador de Regresión Logística.
- **SVD.** TF-IDF más reducción de dimensionalidad con TruncatedSVD, también con Regresión Logística como clasificador final.
- **SVM.** TF-IDF directamente conectado a una Máquina de Vectores de Soporte con kernel lineal.
- **RNA.** TF-IDF seguido de un Perceptrón Multicapa (MLP).

La elección de Regresión Logística como clasificador en los dos primeros modelos no es casual: su entrenamiento es rápido, devuelve probabilidades calibradas de forma natural y sirve como referencia honesta frente a los modelos más complejos (SVM y MLP). Comparar entre sí estas cuatro combinaciones permite observar qué aporta realmente cada componente y dónde compensa el coste adicional.

3.1. Clasificación supervisada en texto clínico

La asignación de una especialidad médica a partir de los síntomas descritos por un paciente se plantea como un problema de clasificación supervisada multiclase [27]. Sea $\mathcal{X} \subseteq \mathbb{R}^d$ el espacio de entrada, formado por las representaciones vectoriales de los textos, y sea $\mathcal{Y} = \{y_1, y_2, \ldots, y_K\}$ el conjunto discreto de las K especialidades médicas posibles [18].

El sistema busca aprender una función $f : \mathcal{X} \to \mathcal{Y}$ que minimice la pérdida empírica regularizada:

$$\hat{h} = \arg\min_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(h(x_i), y_i) + \lambda \Omega(h)$$

Aquí $\mathcal{L}$ mide el error entre la predicción y la etiqueta real, mientras que $\Omega(h)$ penaliza la complejidad del modelo con un peso $\lambda > 0$. El término de regularización es importante en este dominio porque los pacientes describen los mismos síntomas con vocabularios muy distintos, lo que aumenta el riesgo de sobreajuste si el modelo memoriza palabras concretas en lugar de aprender patrones generales [15].

3.2. Preprocesamiento lingüístico y normalización

Antes de poder aplicar cualquier modelo, el texto libre debe transformarse en datos estructurados. Las descripciones que escribe un paciente contienen mayúsculas, signos, faltas de ortografía y palabras vacías que no aportan información clínica útil. El preprocesado reduce esta variabilidad y, de paso, recorta la dimensionalidad del problema [31, 13].

El pipeline aplica las siguientes etapas:

Normalización y tokenización El texto se pasa a minúsculas y se eliminan los caracteres no alfanuméricos mediante una expresión regular que conserva las letras del alfabeto español, incluyendo tildes y la ñ. Mantener los caracteres acentuados antes del stemming es importante: el algoritmo Snowball para español está diseñado para procesar las palabras tal y como aparecen en el idioma, y eliminar las tildes antes de tiempo distorsiona la raíz que termina extrayendo. Tras la

limpieza, la cadena se trocea en una secuencia de unidades léxicas $\mathcal{T} = \{t_1, t_2, \ldots, t_m\}$ [13].

Filtrado de stop-words Se descartan determinantes, conjunciones y términos generales que aparecen en casi cualquier descripción, usando la lista de stopwords del español de NLTK. Estas palabras inflan los vectores sin ayudar a distinguir entre especialidades.

Stemming con Snowball Las palabras se reducen a su raíz mediante el algoritmo Snowball para español [30]. Frente a la lematización, el Stemming es más barato computacionalmente y se integra mejor en pipelines ligeros, lo que encaja con el enfoque de IA Frugal del proyecto [31].

El resultado de esta fase es una cadena depurada que ya se puede vectorizar. La misma función de preprocesado se aplica tanto al entrenamiento como a la predicción en tiempo real, lo que evita desajustes entre cómo se entrenó el modelo y cómo se le pasa el texto del paciente cuando se usa la aplicación.

3.3. Representación vectorial mediante TF-IDF

Una vez normalizado el texto, hace falta convertirlo en vectores numéricos que los modelos puedan procesar. El proyecto utiliza TF-IDF [34], una representación clásica con dos ventajas para este caso: es barata de calcular y resulta fácil de interpretar, ya que cada componente del vector se corresponde con un término concreto del vocabulario [2].

El peso $w_{i,j}$ del término t_i en el documento d_j se calcula como el producto de la frecuencia del término en el documento por la frecuencia inversa documental:

$$w_{i,j} = tf(t_i, d_j) \cdot idf_{smooth}(t_i, D)$$

La implementación se apoya en la versión por defecto de `TfidfVectorizer` de scikit-learn, que aplica dos refinamientos sobre el esquema clásico [34]:

- **IDF suavizado:** $idf_{smooth}(t_i, D) = \log((N+1)/(df(t_i)+1)) + 1$. Da más peso a los términos poco habituales, que en general son los más

útiles para discriminar entre especialidades, y evita divisiones por cero al añadir el documento ficticio que aporta el "+1".

- **Normalización L2:** los vectores resultantes se ajustan a norma unitaria. Así la longitud del texto no condiciona la clasificación, algo importante cuando un paciente puede escribir tres palabras o un párrafo entero.

El tamaño máximo del vocabulario se acota con el parámetro `max_features`: 5.000 términos en los modelos basados en Regresión Logística (TF-IDF base y SVD) y un rango más reducido (1.000 a 5.000) en los que se exploran mediante búsqueda de hiperparámetros (SVM y RNA). En cuanto a los n-gramas, los modelos SVM y RNA combinan unigramas y bigramas para capturar combinaciones clínicas como "insuficiencia renal" o "dolor agudo", mientras que los modelos TF-IDF base y SVD se quedan en unigramas. Esta diferencia es deliberada: los bigramas multiplican el tamaño del vocabulario y solo compensan cuando el clasificador puede aprovechar esa información extra sin penalizar el tiempo de entrenamiento.

Conviene anotar una característica importante de esta representación: la matriz resultante es dispersa (*sparse*), ya que cada documento solo contiene una pequeña fracción del vocabulario total. `TfidfVectorizer` la almacena en formato CSR de SciPy en lugar de como matriz densa, lo que reduce considerablemente el consumo de memoria y acelera las operaciones matriciales sobre ella. Esta propiedad es la razón por la que el SVM lineal del proyecto puede entrenar directamente sobre la salida del TF-IDF sin necesidad de reducir antes la dimensionalidad.

3.4. Reducción de dimensionalidad con TruncatedSVD

La matriz TF-IDF suele ser muy ancha: cada columna es un término del vocabulario y la mayoría de las entradas son cero. Para trabajar con un espacio más manejable se aplica TruncatedSVD, que proyecta la matriz original $X \in \mathbb{R}^{d \times n}$ en un subespacio de rango reducido $k < r$ [21]. En la implementación se ha fijado $k = 100$ componentes, un valor que ofrece un buen compromiso entre capacidad expresiva y coste de cálculo para el tamaño del corpus de entrenamiento.

El teorema de Eckart-Young-Mirsky garantiza que la aproximación truncada $X_k = U_k \Sigma_k V_k^T$ es la mejor representación posible de rango k. Esta

idea conecta con el LSA: las componentes resultantes capturan asociaciones semánticas globales entre términos y documentos, lo que ayuda con problemas de sinonimia habituales en lenguaje médico [21, 18]. En la práctica, dos pacientes pueden describir el mismo cuadro con palabras distintas (por ejemplo, "ahogo" frente a "disnea") y el sistema sigue reconociéndolos como casos próximos en el espacio reducido.

Sobre las 100 componentes resultantes se entrena un clasificador de Regresión Logística. Esta combinación tiene una propiedad útil: la regresión logística devuelve probabilidades calibradas de forma directa, lo que permite mostrar al usuario un porcentaje de confianza junto con la especialidad sugerida y, llegado el caso, derivar a atención general cuando la confianza queda por debajo de un umbral. Comparar este modelo con el TF-IDF base, que usa el mismo clasificador pero sin reducción, deja ver cuánto aporta realmente la SVD en este corpus.

3.5. Clasificación mediante Máquinas de Vector de Soporte

El clasificador SVM se entrena directamente sobre la representación TF-IDF, sin pasar antes por la reducción con TruncatedSVD. Esto es deliberado: las SVM lineales rinden bien en espacios de alta dimensionalidad y matrices dispersas, así que no hay un beneficio claro en reducir el espacio antes de entrenar [5, 27]. Mantener la matriz original también permite contrastar este modelo con el basado en SVD y comprobar si la reducción aporta o no en este corpus concreto.

El SVM busca el hiperplano que separa las clases con el mayor margen posible. El problema de optimización con margen blando se formula como:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

El parámetro C controla el equilibrio entre maximizar el margen y permitir errores de clasificación: un C alto penaliza mucho los fallos y puede llevar a sobreajuste, mientras que un C bajo tolera más errores a cambio de un modelo más general. Una propiedad útil del modelo es que la decisión final solo depende de los vectores de soporte, lo que mantiene el coste de inferencia bajo aunque el corpus de entrenamiento sea grande [5].

En la implementación se utiliza un kernel lineal, que es el más adecuado cuando la representación de entrada ya tiene muchas dimensiones. Además, el clasificador se entrena con el flag de probabilidad activado para poder devolver al usuario un porcentaje de confianza junto con la especialidad predicha, no solo la etiqueta ganadora. Conviene matizar este punto: las SVM, por construcción, no devuelven probabilidades sino distancias al hiperplano de separación. Las probabilidades se obtienen mediante una calibración posterior sobre las puntuaciones del clasificador, que en scikit-learn se implementa mediante el método de Platt. Esto añade un coste de entrenamiento (la calibración requiere validación cruzada interna), pero es la forma estándar de poder ofrecer al usuario una confianza interpretable en porcentaje.

3.6. Clasificación mediante Perceptrón Multicapa

Junto al SVM, el sistema incorpora una Red Neuronal Artificial (RNA) de tipo MLP para capturar relaciones no lineales entre síntomas que un clasificador lineal no puede modelar [28]. En la aplicación y en el resto de la memoria este componente se referencia como RNA, ya que el MLP es la arquitectura concreta de red neuronal artificial que se ha empleado.

El MLP es más caro de entrenar que un SVM lineal, así que el coste se acota por dos vías. La primera es limitar el vocabulario del TF-IDF mediante el parámetro `max_features` (entre 1.000 y 2.000 términos en la búsqueda de hiperparámetros), lo que reduce la dimensión efectiva de la entrada a la red. La segunda es activar la parada temprana (*early stopping*), que detiene el entrenamiento cuando la red deja de mejorar sobre un conjunto de validación interno, evitando iteraciones innecesarias [5, 6].

Para que la red no memorice el conjunto de entrenamiento, se usa el parámetro de regularización Alpha (α), que añade una penalización L2 sobre los pesos [15]. Este control de la complejidad ayuda a generalizar mejor a los textos nuevos del paciente, que casi nunca se parecen literalmente a los de entrenamiento [11].

3.7. Optimización de hiperparámetros y evaluación

Los modelos descritos son sensibles a su configuración: el número de componentes del SVD, el valor de C en el SVM o el tamaño de las capas ocultas en el MLP afectan claramente al rendimiento. Para no fijar estos valores a ojo, en los modelos más complejos se aplica GridSearchCV [6].

Búsqueda de hiperparámetros con GridSearchCV

La búsqueda se aplica sobre los modelos SVM y RNA, que son los que tienen más hiperparámetros relevantes y donde una mala elección penaliza más el resultado. Las combinaciones exploradas son las siguientes:

- **SVM:** valores de $C \in \{0{,}1; 1; 10\}$, `max_features` del TF-IDF en $\{1{,}000;\ 3{,}000;\ 5{,}000\}$ y `min_df` en $\{1, 2\}$.
- **RNA:** arquitecturas con una capa oculta de 64 neuronas o dos capas (128 y 64), valores de $\alpha \in \{0{,}0001;\ 0{,}001\}$, funciones de activación `relu` y `tanh`, y `max_features` del TF-IDF en $\{1{,}000;\ 2{,}000\}$.

Cada combinación se evalúa mediante validación cruzada estratificada con 5 particiones. Estratificar es importante en este caso porque el corpus tiene desbalance entre especialidades (algunas categorías agrupan muchos más casos que otras), y un reparto aleatorio podría dejar particiones sin ejemplos de las clases minoritarias [27].

Los modelos TF-IDF base y SVD prescinden de GridSearchCV y se entrenan con un único reparto entrenamiento/prueba al 80/20. Esta decisión es coherente con el papel que cumplen en el sistema: actúan como modelos de referencia frente a SVM y RNA, no como candidatos a producción, y entrenarlos sin búsqueda de hiperparámetros mantiene la comparativa razonable en tiempo de cómputo.

Métricas de evaluación

A partir de la matriz de confusión se calculan las métricas habituales en clasificación supervisada [18, 6]:

Accuracy Proporción de aciertos sobre el total. Es la métrica que se usa como criterio de selección durante la búsqueda de hiperparámetros, por

su interpretación directa y porque, en este corpus, la diferencia entre los modelos suele ser lo bastante clara como para no requerir métricas más complejas. Su limitación conocida es que, ante desbalance fuerte, un modelo que predice siempre la clase mayoritaria puede aparentar buen rendimiento sin servir para nada.

Precision De los casos que el modelo etiqueta como una especialidad concreta, qué proporción acierta. Penaliza los falsos positivos.

Recall (sensibilidad) De los casos que realmente pertenecen a una especialidad, qué proporción detecta el modelo. Penaliza los falsos negativos, que en triaje clínico son especialmente sensibles.

F1-macro Media aritmética de los F1 por clase. Se reporta como métrica complementaria porque trata por igual a todas las especialidades, sin importar cuántos ejemplos haya de cada una [18], y permite detectar cuándo un modelo con buena *accuracy* en realidad falla en las clases minoritarias.

3.8. Decisión con umbral de confianza y derivación a atención general

Los modelos no devuelven solo una etiqueta: devuelven una distribución de probabilidad sobre todas las especialidades posibles. La aplicación toma la especialidad con mayor probabilidad y muestra al paciente esa probabilidad como porcentaje de confianza. Cuando el porcentaje queda por debajo de un umbral mínimo, el sistema no se aferra a la predicción: pide al paciente síntomas adicionales y, si después de varios intentos la confianza sigue siendo baja, deriva el caso a atención general y guarda los síntomas en un log para que un administrador los revise más adelante.

Este planteamiento se conoce como clasificación con umbral de confianza: el modelo puede declinar una predicción incierta en lugar de forzar una respuesta posiblemente errónea. Los clasificadores devuelven una puntuación de confianza entre 0 y 1, y se descartan las predicciones cuya puntuación queda por debajo de un umbral establecido por el sistema [14]. En triaje clínico tiene sentido directo, ya que un infra-triaje (no derivar a tiempo a la especialidad correcta) puede tener consecuencias serias, y vale más reconocer la duda que asignar mal una cita.

3.9. Captura de síntomas por voz

La interfaz de paciente acepta la entrada por dos canales: texto escrito directamente en el formulario o dictado por voz mediante un componente de captura de audio del navegador, que transcribe lo dictado a texto en español. La transcripción se delega al motor de reconocimiento de voz del navegador, por lo que no forma parte del modelo de clasificación desarrollado en este proyecto. Una vez transcrita, la cadena pasa por exactamente la misma cadena de preprocesamiento que cualquier otro texto, así que el modelo no distingue si los síntomas vinieron del teclado o del micrófono.

Esta separación entre el canal de entrada y el núcleo de clasificación se ha mantenido a propósito. Permite añadir más canales en el futuro (por ejemplo, transcripciones automáticas de llamadas telefónicas para gestión de citas) sin tocar la parte de modelado. La transcripción de voz no se considera una pieza central del sistema y por eso no se profundiza en su funcionamiento, pero conviene mencionar que existe porque ya está integrada en la aplicación.

3.10. IA Frugal y Edge AI

Las decisiones técnicas anteriores no son neutras: persiguen un sistema útil con el mínimo gasto de recursos. Esta idea se conoce como IA Frugal y está recogida en marcos como AFNOR SPEC 2314 [2, 11].

En este proyecto se traduce en dos aspectos concretos:

- **Eficiencia técnica.** TF-IDF es una representación barata y, en el modelo SVD, TruncatedSVD comprime esa representación a 100 componentes antes de clasificar. Los clasificadores escogidos (Regresión Logística, SVM lineal y un MLP pequeño) se entrenan en tiempos razonables sobre hardware modesto y caben sin problema en un equipo de consulta médica estándar [2].

- **Privacidad y autonomía.** Toda la inferencia ocurre en el dispositivo (Edge AI), sin enviar los síntomas del paciente a un servidor externo. Esto cumple con el principio de privacidad por diseño y, además, hace que el sistema funcione aunque la consulta no tenga conexión a internet [11].

3.11. Conclusiones del capítulo

La combinación de un preprocesamiento común y cuatro modelos con configuraciones distintas (TF-IDF con Regresión Logística, TF-IDF con TruncatedSVD y Regresión Logística, TF-IDF con SVM lineal y TF-IDF con MLP) cumple con los requisitos del proyecto: la arquitectura es ligera, funciona sin conexión y permite trazar las decisiones del modelo término a término. La inclusión de un umbral de confianza con derivación a atención general añade una capa de prudencia que un sistema de producción real necesita.

A cambio, hereda las limitaciones de los modelos basados en bolsa de palabras. La negación ("no tengo fiebre") no se distingue bien de la afirmación, los matices semánticos se pierden y hay ambigüedades clínicas que un sistema sin conocimiento médico explícito difícilmente puede resolver. Estas carencias se han asumido de forma consciente y quedan apuntadas como líneas de mejora para futuras versiones.

4. Técnicas y herramientas

Esta parte de la memoria tiene como objetivo presentar las técnicas metodológicas y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. Si se han estudiado diferentes alternativas de metodologías, herramientas, bibliotecas se puede hacer un resumen de los aspectos más destacados de cada alternativa, incluyendo comparativas entre las distintas opciones y una justificación de las elecciones realizadas. No se pretende que este apartado se convierta en un capítulo de un libro dedicado a cada una de las alternativas, sino comentar los aspectos más destacados de cada opción, con un repaso somero a los fundamentos esenciales y referencias bibliográficas para que el lector pueda ampliar su conocimiento sobre el tema.

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros3, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Acrónimos

CoT *Cadena de pensamiento*. 3

GUI *Interfaz Gráfica de Usuario*. 3

RNA Red Neuronal Artificial. 16

SVD *Singular Value Decomposition*. 5, 7, 9

SVM *Support Vector Machine*. 5, 7, 9

TF-IDF *Term Frequency–Inverse Document Frequency*. 5, 6, 7, 9, 11, 13, 14, 15, 16, 19, 20

Glosario

GridSearchCV Búsqueda de rejilla con validación cruzada (*Grid Search Cross-Validation*). Procedimiento de exploración estructurada sobre un conjunto discreto de hiperparámetros para encontrar la combinación óptima. 17

LSA Análisis Semántico Latente (*Latent Semantic Analysis*). Técnica de procesamiento de lenguaje natural que utiliza SVD para identificar patrones en las relaciones entre términos y conceptos. 15

MLP Perceptrón Multicapa (*Multi-layer Perceptron*). Red neuronal artificial feedforward que captura relaciones no lineales mediante capas ocultas y entrenamiento por retropropagación. 11, 16, 19, 20

Naive Bayes Familia de clasificadores probabilísticos basada en el teorema de Bayes y en la asunción de independencia condicional entre características. 6

Recall (sensibilidad) Métrica que cuantifica la proporción de casos reales de una clase identificados correctamente por el modelo; se calcula como TP/(TP+FN). 7, 18

Regresión logística Modelo de clasificación supervisada que estima la probabilidad de pertenencia a una clase a partir de una combinación lineal de características. 6

stand alone Propiedad de un sistema que puede ejecutarse de forma autónoma en un entorno local, sin depender de servicios externos o conexión a internet. 5

Stemming Proceso lingüístico de reducción de una palabra a su raíz o tallo (*stem*). Técnica de bajo coste computacional comparada con la lematización. 13

TruncatedSVD Técnica de reducción de dimensionalidad que aproxima la descomposición en valores singulares conservando solo k componentes principales; se aplica frecuentemente sobre matrices TF-IDF para obtener representaciones latentes. 6, 11, 14, 15, 19, 20

Bibliografía

[1] ACM SIGKDD Explorations. Frugal ai: Introduction, concepts, development and open questions. https://kdd.org/exploration_files/p72-Frugal_AI.pdf, 2020. [Internet; accedido 29-enero-2026].

[2] AFNOR. Afnor spec 2314: Marco general para la ia frugal. Technical report, Association Française de Normalisation, 2024. [Internet; consultado mayo-2026].

[3] arXiv. A comparative study on tf-idf feature weighting method and its analysis using unstructured dataset. https://arxiv.org/abs/2308.04037, 2023. [Internet; accedido 29-enero-2026].

[4] Clarifai. Edge vs cloud ai: Key differences, benefits & hybrid future. https://www.clarifai.com/blog/edge-vs-cloud-ai, 2024. [Internet; accedido 29-enero-2026].

[5] Codemia. Svm frente a la red neuronal mlp: Comparación de rendimiento y precisión. 2024. [Internet; consultado mayo-2026].

[6] DergiPark. Optimización de hiperparámetros y entrenamiento de modelos para diagnóstico clínico. 2024. [Internet; consultado mayo-2026].

[7] DhiWise. Build ada health app clone: A detailed development guide. https://www.dhiwise.com/post/building-ai-health-assistant-like-ada-health-app, 2024. [Internet; accedido 29-enero-2026].

[8] DIVA Portal. Automated triage in digital primary care. https://www.diva-portal.org/smash/get/diva2:1452894/FULLTEXT01.pdf, 2020. [Internet; accedido 29-enero-2026].

[9] DIVA Portal. Bachelor degree project: Svm vs llm: A comparative analysis on text classification. https://lnu.diva-portal.org/smash/get/diva2:1975221/FULLTEXT01.pdf, 2024. [Internet; accedido 29-enero-2026].

[10] Fierce Healthcare. Once a 'unicorn,' babylon health sells uk business to emed via bankruptcy process. https://www.fiercehealthcare.com/digital-health/babylon-closes-us-business-lays-employees-after-mindmaze-take-private-deal-collapses, 2023. [Internet; accedido 29-enero-2026].

[11] Frugal AI Hub. Ia frugal: Impulsando la innovación sostenible e inclusiva, 2025. [Internet; consultado mayo-2026].

[12] Hospitalogy. The downfall of babylon health. https://hospitalogy.com/articles/2023-05-16/downfall-of-babylon-health/, 2023. [Internet; accedido 29-enero-2026].

[13] IBM. Stemming vs lematización en el procesamiento de texto, 2024. [Internet; consultado mayo-2026].

[14] IBM. ¿qué son los modelos de clasificación?, 2024. [Internet; consultado 9-mayo-2026].

[15] Interactive Chaos. Regularización l2 en redes neuronales, 2023. [Internet; consultado mayo-2026].

[16] JMIR AI. Traditional machine learning, deep learning, and bert (large language model) approaches for predicting hospitalizations from nurse triage notes: Comparative evaluation of resource management. https://ai.jmir.org/2024/1/e52190, 2024. [Internet; accedido 29-enero-2026].

[17] Konvoy Ventures. Local vs cloud ai. https://www.konvoy.vc/newsletters/local-vs-cloud-ai, 2024. [Internet; accedido 29-enero-2026].

[18] P. López-Úbeda et al. Algoritmos de aprendizaje automático tradicionales para la clasificación de texto médico. *MDPI Computers*, 15(1), 2025. [Internet; consultado mayo-2026].

[19] MDPI. Recent advances in large language models for healthcare. https://www.mdpi.com/2673-7426/4/2/62, 2024. [Internet; accedido 29-enero-2026].

[20] Medium. Natural language processing: Applying svd on term-document matrices. https://medium.com/@RobuRishabh/understanding-term-document-matrices-and-singular-value-decomposition-svd-in-nlp-3e08789073f6, 2020. [Internet; accedido 29-enero-2026].

[21] Medium. Descomposición en valores singulares en pln explicada, 2023. [Internet; consultado mayo-2026].

[22] Ministerio de Sanidad (España). Informe barómetro sanitario 2024. https://www.sanidad.gob.es/estadEstudios/estadisticas/BarometroSanitario/Barom_Sanit_2024/BS_2024_Totales/Informe_BS_2024.pdf, 2024. [Internet; accedido 29-enero-2026].

[23] Hai Nguyen, Andras Meczner, Krista Burslam-Dawe, and Benedict Hayhoe. Triage errors in primary and pre–primary care. *Journal of Medical Internet Research*, 24(6):e37209, 2022.

[24] Oxford Academic. Large language models are less effective at clinical prediction tasks than locally trained machine learning models. https://academic.oup.com/jamia/article/32/5/811/8064348, 2025. [Internet; accedido 29-enero-2026].

[25] pharmaphorum. Study finds big differences between top symptom checker apps. https://pharmaphorum.com/news/study-finds-big-differences-between-top-symptom-spotting-apps, 2020. [Internet; accedido 29-enero-2026].

[26] Revista Emergencias. La saturación en los servicios de urgencias hospitalarios. https://revistaemergencias.org/wp-content/uploads/2023/08/Emergencias-2015_27_2_113-120-120.pdf, 2015. [Internet; accedido 29-enero-2026].

[27] Scikit-learn. Algoritmos multiclase y multietiqueta, 2024. [Internet; consultado mayo-2026].

[28] Scikit-learn. Modelos de redes neuronales (supervisados): Perceptrón multicapa, 2024. [Internet; consultado mayo-2026].

[29] SMILE (EU). Frugal ai: sustainable, responsible and high-performance artificial intelligence. https://smile.eu/en/publications-and-events/frugal-ai-sustainable-responsible-and-high-performance-artificial, 2021. [Internet; accedido 29-enero-2026].

[30] Snowball. El algoritmo de stemming snowball, 2022. [Internet; consultado mayo-2026].

[31] Snowflake. Fundamentos de procesamiento de lenguaje natural, 2024. [Internet; consultado mayo-2026].

[32] TechTarget (PatientEngagement). Average patient appointment wait time is 31 days in 2025. https://www.techtarget.com/patientengagement/news/366624919/Average-patient-appointment-wait-time-is-31-days-in-2025, 2025. [Internet; accedido 29-enero-2026].

[33] Univadis. Informe del sistema nacional de salud 2024: aspectos relevantes de la atención primaria. https://www.univadis.es/viewarticle/informe-del-sistema-nacional-sud-2024-aspectos-relevantes-2025a1000yrm?uuid=1861de44-d0eb-4f80-9226-e0f4aae4e000, 2024. [Internet; accedido 29-enero-2026].

[34] Wikipedia. Tf–idf: Frecuencia de término – frecuencia inversa de documento, 2025. [Internet; consultado mayo-2026].